CHEAT SHEET · JS INTERVIEW #12 @specsycoder

sort() on numbers

Why [10, 1, 2].sort() gives [1, 10, 2]

sort.js

```
1  [10, 1, 2].sort();
```

output 1 10 2

Why it happens

01 Strings by default

With no compare function, sort converts every item to a string.

02 Dictionary order

Strings are compared character by character, left to right.

03 "10" vs "2"

First characters: "1" vs "2". "1" is smaller, so "10" goes first.

04 It mutates

sort changes the original array AND returns it.

How it compares

how-it-compares.js

```
1  // sort turns items into strings first
2  // "10" vs "2" -> compare "1" vs "2"
3  // "1" comes first, so "10" comes before "2"
```

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

@specsycoder

The compare function

How a - b decides the order

fix.js

```js
[10, 1, 2].sort((a, b) => a - b); // [1, 2, 10]
[10, 1, 2].sort((a, b) => b - a); // [10, 2, 1]
arr.toSorted((a, b) => a - b); // new array
```

compare returns	meaning
negative	a goes before b
positive	b goes before a
0	keep their order

expression	result
["b", "a", "C"].sort()	["C", "a", "b"]
[5, 25, 100].sort()	[100, 25, 5]
names.sort((a, b) => a.localeCompare(b))	A to Z, any case

Traps to mention

- **Uppercase sorts before lowercase** by default ("C" before "a").
- **sort mutates.** Use `toSorted()` or `[...arr].sort()` to keep the original.

Your 20-second answer

"By default, sort converts items to strings and compares them in dictionary order, so '10' comes before '2'. For numbers I pass a compare function, `(a, b) => a - b`. A negative result puts a first. And sort mutates the array, so I use toSorted when I need a copy."